

MOFA+: model, priors, and variational algorithm

1. Overview

MOFA+ (Argelaguet et al., *Genome Biology*, 2020) is a multi-view, multi-group factor analysis model fit by mean-field variational inference. It extends the original MOFA (Argelaguet et al., *MSB*, 2018) with an explicit *group* axis (so factors can be active in some groups and not others), per-feature noise precisions, and stochastic / GPU options.

In MOFA+ terminology applied to our ModuleFinder setup:

- **views** $m = 1, \dots, M$ — contexts A, B, C
- **groups** $g = 1, \dots, G$ — a single group **all**
- **samples** $n = 1, \dots, N_g$ — *genes* (5000) after we transposed
- **features** $d = 1, \dots, D_m$ — *perturbations* (60) per context
- **factors** $k = 1, \dots, K$ — the latent dimension MOFA fits

This document spells out the generative model, the priors, the variational inference scheme, and how the variance-explained quantities we use downstream are computed.

2. Generative model

Let $Y^{(m,g)} \in \mathbb{R}^{N_g \times D_m}$ be the observed matrix for view m , group g . MOFA+ assumes:

$$Y_{n,d}^{(m,g)} = \sum_{k=1}^K Z_{n,k}^{(g)} W_{d,k}^{(m)} + \varepsilon_{n,d}^{(m,g)}$$

with Gaussian noise

$$\varepsilon_{n,d}^{(m,g)} \sim \mathcal{N}(0, \tau_{m,d}^{-1})$$

so each feature in each view has its own precision $\tau_{m,d}$ (heteroscedastic across features, shared across samples). In matrix form,

$$Y^{(m,g)} = Z^{(g)} (W^{(m)})^\top + E^{(m,g)}.$$

- $Z^{(g)} \in \mathbb{R}^{N_g \times K}$ — factor matrix, **shared across views** within group g
- $W^{(m)} \in \mathbb{R}^{D_m \times K}$ — loading matrix, **shared across groups** within view m

The *same* K factors appear in every view $\times$ group combination, but ARD priors (below) drive entire (factor, view) or (factor, group) blocks to zero so unused combinations are effectively pruned.

Non-Gaussian likelihoods (Bernoulli, Poisson) are supported via Jaakkola-style lower bounds; we use the Gaussian case throughout.

3. Priors

3.1 Factors: per-(group, factor) ARD

$$Z_{n,k}^{(g)} \sim \mathcal{N}(0, (\alpha_{k,g}^Z)^{-1}), \quad \alpha_{k,g}^Z \sim \text{Gamma}(a_0, b_0)$$

A separate precision $\alpha_{k,g}^Z$ for each (factor, group) lets the model shut off a factor in a group it doesn't need: $\alpha_{k,g}^Z \rightarrow \infty$ forces $Z_{n,k}^{(g)} \rightarrow 0$ for that group while leaving the factor active in other groups.

3.2 Loadings: per-(view, factor) ARD + spike-and-slab

This is the part that gives MOFA+ its sparsity. For each loading entry:

$$W_{d,k}^{(m)} = S_{d,k}^{(m)} \widehat{W}_{d,k}^{(m)}$$

with

$$\widehat{W}_{d,k}^{(m)} \sim \mathcal{N}(0, (\alpha_{k,m}^W)^{-1}), \quad S_{d,k}^{(m)} \sim \text{Bernoulli}(\theta_k^{(m)})$$

and hyperpriors

$$\alpha_{k,m}^W \sim \text{Gamma}(a_0, b_0), \quad \theta_k^{(m)} \sim \text{Beta}(a_\theta, b_\theta).$$

Two complementary sparsity mechanisms work together:

- **ARD** ($\alpha_{k,m}^W$) — soft, view-level: pushes the entire column $W_{\cdot,k}^{(m)}$ toward zero if factor k is irrelevant in view m . Small $\alpha^W \Rightarrow$ active factor; large $\alpha^W \Rightarrow$ inactive.
- **Spike-and-slab** ($S_{d,k}^{(m)}$) — hard, feature-level: each individual loading can be exactly zero. $\theta_k^{(m)}$ controls the expected density of factor k 's loadings in view m .

Together they give a sparse-and-shrunk loading matrix, which is the mechanism by which MOFA+ identifies a small number of informative factors out of an initially generous K .

3.3 Noise precisions

$$\tau_{m,d} \sim \text{Gamma}(c_0, d_0).$$

Per-feature, per-view (but pooled across samples and groups). This is what lets MOFA+ down-weight noisy features automatically.

4. Variational inference

The exact posterior $p(Z, \widehat{W}, S, \alpha^Z, \alpha^W, \theta, \tau \mid Y)$ is intractable. MOFA+ approximates it with a mean-field variational distribution that factorises across all latent variables:

$$q(Z, \widehat{W}, S, \alpha^Z, \alpha^W, \theta, \tau) = \prod_{g,n,k} q(Z_{n,k}^{(g)}) \prod_{m,d,k} q(\widehat{W}_{d,k}^{(m)}, S_{d,k}^{(m)}) \\ \times \prod_{g,k} q(\alpha_{k,g}^Z) \prod_{m,k} q(\alpha_{k,m}^W) \prod_{m,k} q(\theta_k^{(m)}) \prod_{m,d} q(\tau_{m,d}).$$

Each variational factor is updated by **coordinate ascent** (CAVI): set $q_i(\cdot) \propto \exp\{\mathbb{E}_{q_{-i}}[\log p(\text{data}, \text{latents})]\}$ where q_{-i} is the product of all variational factors except q_i . Because every prior is conjugate to its Gaussian likelihood (or to the appropriate exponential-family local conditional), the optimal variational families are closed-form:

latent	optimal q
$Z_{n,\cdot}^{(g)}$	Multivariate Normal in $\mathbb{R}^K$
$(\widehat{W}_{d,k}^{(m)}, S_{d,k}^{(m)})$	Bernoulli $\times$ Normal product (spike-slab)
$\alpha_{k,g}^Z, \alpha_{k,m}^W, \tau_{m,d}$	Gamma
$\theta_k^{(m)}$	Beta

The objective being maximised is the evidence lower bound (ELBO):

$$\mathcal{L}(q) = \mathbb{E}_q[\log p(Y, \text{latents})] - \mathbb{E}_q[\log q].$$

Convergence is monitored by the relative change of $\mathcal{L}$ between iterations (**convergence_mode** in the API: 'fast' / 'medium' / 'slow' sets the tolerance and the number of contiguous below-tolerance iterations required).

5. Update equations (sketch)

To make the inner loop concrete, here are the per-iteration updates. Write $\langle \cdot \rangle \equiv \mathbb{E}_q[\cdot]$.

5.1 Factor update

For sample n in group g , define the residual sufficient statistics across all views:

$$\Lambda_n^{(g)} = \text{diag}(\langle \alpha_{\cdot,g}^Z \rangle) + \sum_{m=1}^M \sum_{d=1}^{D_m} \langle \tau_{m,d} \rangle \langle W_{d,\cdot}^{(m)} (W_{d,\cdot}^{(m)})^\top \rangle.$$

$$\mu_n^{(g)} = (\Lambda_n^{(g)})^{-1} \sum_{m=1}^M \sum_{d=1}^{D_m} \langle \tau_{m,d} \rangle Y_{n,d}^{(m,g)} \langle W_{d,\cdot}^{(m)} \rangle.$$

Then $q(Z_{n,\cdot}^{(g)}) = \mathcal{N}(\mu_n^{(g)}, (\Lambda_n^{(g)})^{-1})$.

5.2 Loading update (spike-slab)

For feature d in view m , factor k :

1. Conditional on the slab being on ($S = 1$), the optimal Gaussian over $\widehat{W}_{d,k}^{(m)}$ has

$$\hat{v}_{m,d,k} = \frac{1}{\langle \alpha_{k,m}^W \rangle + \langle \tau_{m,d} \rangle \sum_{g,n} \langle (Z_{n,k}^{(g)})^2 \rangle},$$

$$\hat{\mu}_{m,d,k} = \hat{v}_{m,d,k} \langle \tau_{m,d} \rangle \sum_{g,n} \langle Z_{n,k}^{(g)} \rangle \left(Y_{n,d}^{(m,g)} - \sum_{k' \neq k} \langle Z_{n,k'}^{(g)} W_{d,k'}^{(m)} \rangle \right).$$

2. The Bernoulli responsibility for $S_{d,k}^{(m)} = 1$:

$$\log \frac{\langle S_{d,k}^{(m)} \rangle}{1 - \langle S_{d,k}^{(m)} \rangle} = \langle \log \theta_k^{(m)} \rangle - \langle \log(1 - \theta_k^{(m)}) \rangle + \frac{1}{2} \left[\frac{\hat{\mu}_{m,d,k}^2}{\hat{v}_{m,d,k}} - \log(\hat{v}_{m,d,k} \langle \alpha_{k,m}^W \rangle) \right].$$

3. Combined moment: $\langle W_{d,k}^{(m)} \rangle = \langle S_{d,k}^{(m)} \rangle \hat{\mu}_{m,d,k}$.

5.3 Hyperparameter updates

Each Gamma / Beta hyperparameter is updated via its conjugate sufficient statistic. For instance,

$$\alpha_{k,m}^W \mid \cdot \sim \text{Gamma} \left(a_0 + \frac{D_m}{2}, \quad b_0 + \frac{1}{2} \sum_{d=1}^{D_m} \langle (\widehat{W}_{d,k}^{(m)})^2 \rangle \right),$$

so when most of the loadings on factor k in view m are tiny, the rate parameter grows and $\langle \alpha_{k,m}^W \rangle$ blows up — formally this is the ARD pruning step.

Likewise,

$$\tau_{m,d} \mid \cdot \sim \text{Gamma} \left(c_0 + \frac{\sum_g N_g}{2}, \quad d_0 + \frac{1}{2} \sum_{g,n} \langle (Y_{n,d}^{(m,g)} - Z_n^{(g)} W_d^{(m)})^2 \rangle \right).$$

6. Algorithm in pseudocode

Inputs:

```
Y[m,g]           # data per (view, group)
K_init           # initial number of factors (>> expected K_true)
max_iter, tol
```

Initialise:

```
Z, W_hat, S      # PCA on concatenated views, set S = 1
alpha^Z, alpha^W  # set to 1
tau              # set to 1
```

```

theta          # set to 0.5

for t in 1..max_iter:
    # --- Variational E-steps ---
    for g, n:
        update q(Z^{(g)}_{n,:})          # Gaussian, eqs 5.1
    for m, d, k:
        update q(W_hat^{(m)}_{d,k}, S^{(m)}_{d,k})  # spike-slab, eqs 5.2
    update q(alpha^Z), q(alpha^W)          # Gamma
    update q(theta)                        # Beta
    update q(tau)                          # Gamma

    # --- Convergence check ---
    L_t = ELBO(q)
    if |L_t - L_{t-1}| / |L_{t-1}| < tol:
        break

    # --- Optional pruning ---
    drop any factor k with <alpha^W_{k,m}> > 10^6 for all m
    AND <alpha^Z_{k,g}> > 10^6 for all g

return Z, W, alpha^W, alpha^Z, theta, tau, R^2

```

Two practical notes:

- **Initialisation.** MOFA+ initialises Z from PCA on the concatenated views. This dramatically speeds up convergence vs. random init.
- **Pruning.** Factors are explicitly dropped from the model when ARD marks them inactive in every (view, group) combination. This is what produces the “dead” factor label in `factor_summary.csv` downstream.

7. Variance explained

After convergence, MOFA+ reports per-(view, group, factor) coefficient of determination

$$R_{m,g,k}^2 = 1 - \frac{\|Y^{(m,g)} - \langle Z_{\cdot,k}^{(g)} \rangle \langle W_{\cdot,k}^{(m)} \rangle^\top\|_F^2}{\|Y^{(m,g)}\|_F^2}.$$

This is what we read out of `variance_explained/r2_per_factor/<group>` in the HDF5 and use as the “factor active in view” criterion. Because the α^W posteriors are not directly written to disk in mofapy2 0.7.x, we use $1/R^2$ as a proxy for $\langle \alpha^W \rangle$ in our downstream shared-vs-private classification.

A factor with $R_{m,g,k}^2 \approx 0$ in every view is effectively “dead” — pruned by ARD.

8. MOFA+ over the original MOFA

What MOFA+ adds beyond MOFA (2018):

1. **Group axis.** The $\alpha_{k,g}^Z$ ARD lets a factor be active in one group of samples and inactive in another. The original MOFA only had per-view ARD on loadings.
2. **Stochastic VI.** Mini-batch updates of $q(Z)$ enable scaling to $N \gtrsim 10^6$ samples (single-cell datasets).
3. **GPU backend.** `gpu_mode=True` runs the linear-algebra-heavy updates on the GPU via `cupy`.
4. **Improved sparsity initialisation.** Spike-slab indicator S is warm-started so factors don't collapse early.
5. **Sparse-GP option.** Z can be given a spatial / temporal Gaussian process prior (for spatial transcriptomics, time-course data).

Where MOFA+ is conservative compared with deeper non-linear models:

- Everything is **linear** in Z : $Y \approx ZW^\top + \varepsilon$. Non-linear factor-mixing isn't expressible.
 - The mean-field assumption can underestimate posterior variance and occasionally over-prune factors (one of the failure modes corrected by post-hoc inspection of `factor_summary.csv`).
 - Identifiability is up to factor permutation, sign flip, and rotation *within an unconstrained subspace* — ARD partially fixes the rotation by preferring axis-aligned sparse loadings, but you should not read individual factor directions as canonical without inspecting the ARD spectrum.
-

9. References

1. Argelaguet, R. *et al.* (2018). *Multi-Omics Factor Analysis—a framework for unsupervised integration of multi-omics data sets*. **Molecular Systems Biology**, 14:e8124.
2. Argelaguet, R. *et al.* (2020). *MOFA+: a statistical framework for comprehensive integration of multi-modal single-cell data*. **Genome Biology**, 21:111.
3. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*, chapters 9–10 (mean-field VI, ARD).
4. mofapy2 source: <https://github.com/bioFAM/mofapy2>